

SIMATS ENGINEERING

CO1 - ASSESSMENT TOOL 1

Analytical Problem Solving

COURSE NAME: Deep Learning
FACULTY : Dr.Arul Raj A.M

COURSE CODE: MLA0403

COURSE OUTCOME COVERED

CO 1: Learning Algorithms, Maximum Likelihood Estimation (MLE), Building Machine Learning Algorithms, Neural Networks, Artificial Neurons, Perceptron, Multilayer Perceptron (MLP), Forward Propagation, Backpropagation Algorithm, Gradient Descent, Stochastic Gradient Descent (SGD), Mini-Batch Gradient Descent, Curse of Dimensionality. (BTL-4)

CO1 Weightage: 15%

Assessment Tool Weightage: 35%

Duration: 30 minutes

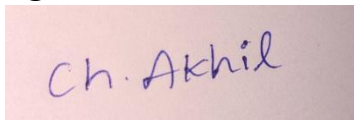
Marks: 25

Code of Conduct:

I Ch.Akhil (192525250) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

A rectangular box containing a handwritten signature in blue ink that reads "Ch. Akhil".

Name of the student: Ch.Akhil

Analytical Problem Solving

1. Learning Algorithms (Optimization Behavior)

A machine learning model is trained using gradient descent on a convex loss function. Initially, the learning rate is set very high, and later it is gradually decreased.

Question:

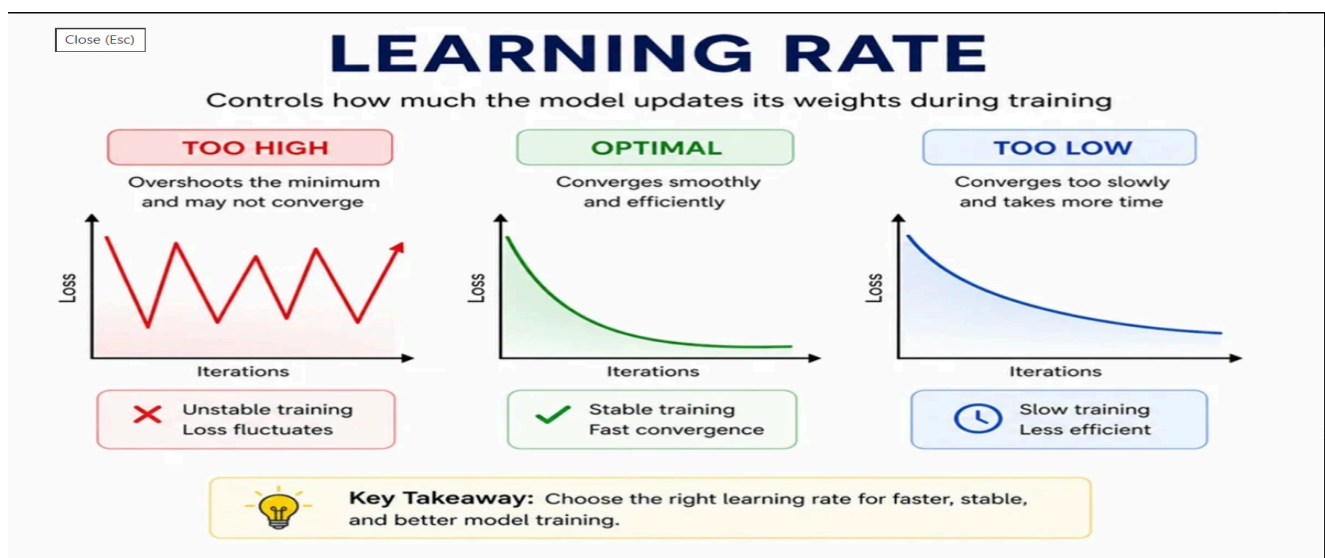
Analyze how the choice of a high initial learning rate followed by decay affects convergence. Under what conditions can this strategy outperform a constant learning rate? Discuss possible risks and justify with reasoning.

Answer:

Gradient descent is an optimization algorithm used to minimize the loss function by updating model parameters in the direction of the negative gradient. When a **high initial learning rate** is used, the model makes large parameter updates, allowing it to move quickly toward the minimum of a convex loss function. This significantly speeds up the early stages of training, especially when the initial parameters are far from the optimal solution. However, if the learning rate is kept high throughout training, the algorithm may overshoot the minimum, causing oscillations or even divergence.

To overcome this problem, the learning rate is **gradually decreased (learning rate decay)** as training progresses. A smaller learning rate in the later stages enables the model to make fine adjustments, reducing oscillations and improving convergence accuracy. This combination of a high initial learning rate followed by gradual decay provides a balance between **fast learning** and **stable optimization**.

This strategy often outperforms a constant learning rate when the loss function is convex, the dataset is large, and training requires many iterations. A constant low learning rate leads to slow convergence, while a constant high learning rate may never settle at the optimum. In contrast, learning rate decay allows the model to learn rapidly at first and then refine the solution with smaller, more precise updates. As a result, it usually converges faster and achieves better final performance.



Despite its advantages, this approach also has risks. If the initial learning rate is excessively high, the optimizer may skip over the optimal point repeatedly, making training unstable or causing the loss to increase instead of decrease. Likewise, if the learning rate is reduced too quickly, updates become too small, slowing learning and preventing the model from reaching the best possible solution. Therefore, selecting an appropriate initial learning rate and an effective decay schedule is crucial.

In conclusion, using a high initial learning rate followed by gradual decay is an effective optimization strategy for convex loss functions. It combines the benefits of rapid initial convergence with stable and accurate final convergence, often outperforming a constant learning rate when properly tuned. However, careful selection of the learning rate and decay schedule is essential to avoid overshooting, divergence, or slow convergence.

2. Maximum Likelihood Estimation (MLE)

Suppose you are given a dataset generated from a Gaussian distribution with unknown mean μ and known variance σ^2

Question:

Derive the Maximum Likelihood Estimator for μ , and analytically explain how the estimate changes when:

- The dataset contains outliers
 - The sample size increases
- What does this imply about the robustness of MLE?

Answer:

Maximum Likelihood Estimation (MLE) is a statistical method used to estimate unknown parameters of a probability distribution by finding the values that maximize the likelihood of observing the given data. For a Gaussian (Normal) distribution with **unknown mean (μ)** and **known variance (σ^2)**, the likelihood function is constructed from the observed samples. By taking the logarithm of the likelihood function, differentiating it with respect to μ , and setting the derivative to zero, the Maximum Likelihood Estimator is obtained as:

$$\hat{\mu}_{MLE} = \frac{1}{n} \sum_{i=1}^n x_i$$

Thus, the MLE of the mean is simply the **sample average**.

When the dataset contains **outliers**, the estimate is strongly affected because the sample mean gives equal weight to every observation. A few extremely large or small values can pull the estimated mean away from the true center of the distribution, resulting in a biased estimate. Therefore, MLE based on the Gaussian assumption is **sensitive to outliers** and is not considered robust in such situations.

As the **sample size increases**, the estimate becomes more stable and accurate. According to the Law of Large Numbers, the sample mean converges to the true population mean as more observations are collected. In addition, the variance of the estimator decreases with increasing sample size, making the estimate more reliable. Although occasional outliers may still influence the estimate, their relative effect becomes smaller in a large dataset.

Implication for Robustness:

MLE provides efficient and accurate estimates when the data truly follow a Gaussian distribution and contain few or no outliers. However, because the estimator is the sample mean, it is **not robust to outliers**. In practical applications where noisy or abnormal observations are expected, robust estimation methods such as the **median** or **trimmed mean** are often preferred. Overall, MLE offers high statistical efficiency for clean data but reduced robustness in the presence of extreme values.

3. Building a Machine Learning Algorithm (Model Selection)

You are designing a classification algorithm for a dataset with 10,000 features but only 500 training samples.

Question:

Analyze the challenges involved in training such a model. Propose a strategy (algorithm pipeline) to handle this situation and justify each step (e.g., feature selection, regularization, dimensionality reduction). Explain how your approach mitigates overfitting.

Answer:

Training a classification model with 10,000 features but only 500 training samples is a challenging problem because the number of features is much larger than the number of observations. This situation leads to the curse of dimensionality, where the model can easily memorize the training data instead of learning meaningful patterns. As a result, the model suffers from overfitting, high computational cost, and poor performance on unseen data. In addition, many features may be irrelevant or highly correlated, increasing noise and reducing model accuracy.

A suitable machine learning pipeline begins with data preprocessing, where missing values are handled, duplicate records are removed, and all features are standardized. Standardization ensures that features with different scales contribute equally during training. Next, feature selection is applied using techniques such as correlation analysis, mutual information, or L1 (Lasso) regularization to remove irrelevant and redundant features. This reduces the dimensionality while preserving the most informative variables.

After feature selection, dimensionality reduction techniques such as Principal Component Analysis (PCA) can be used to transform the remaining features into a smaller set of uncorrelated components. This reduces computational complexity and removes noise while retaining most of the important information. The reduced feature set is then used to train a classifier such as Logistic Regression with L2 regularization or a Support Vector Machine (SVM) with an appropriate kernel. Regularization adds a penalty to large model weights, preventing the model from becoming overly complex.

Finally, k-fold cross-validation is performed to evaluate the model and tune hyperparameters. Cross-validation provides a reliable estimate of model performance and helps select the best model without overfitting to the training data.

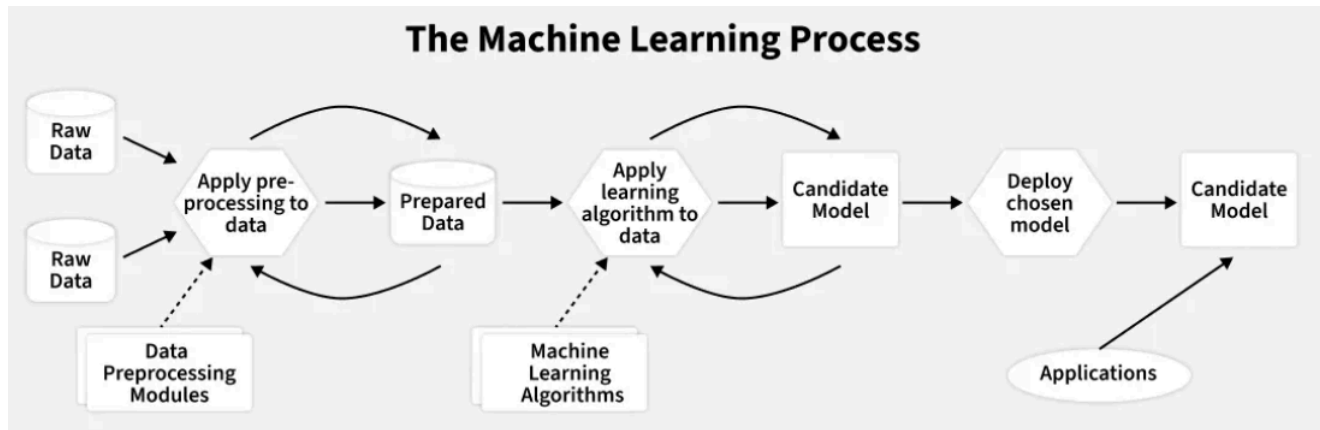
How the Approach Mitigates Overfitting:

- Feature Selection: Removes irrelevant and noisy features.
- Dimensionality Reduction (PCA): Compresses data into informative components and reduces complexity.
- Regularization (L1/L2): Prevents excessively large weights and simplifies the model.
- Cross-Validation: Ensures the model generalizes well to unseen data.

Conclusion:

By combining preprocessing, feature selection, dimensionality reduction, regularization, and

cross-validation, the algorithm becomes more efficient, reduces overfitting, improves generalization, and achieves better classification performance even when the dataset has 10,000 features but only 500 training samples.



4. Neural Networks (Multilayer Perceptron - MLP)

Consider a Multilayer Perceptron with one hidden layer using sigmoid activation functions.

Question:

Explain analytically why adding more hidden neurons increases the representational power of the network. Then, reason why this does not always lead to better performance. Support your answer using concepts like overfitting and generalization.

Answer:

Neural Networks (Multilayer Perceptron - MLP)

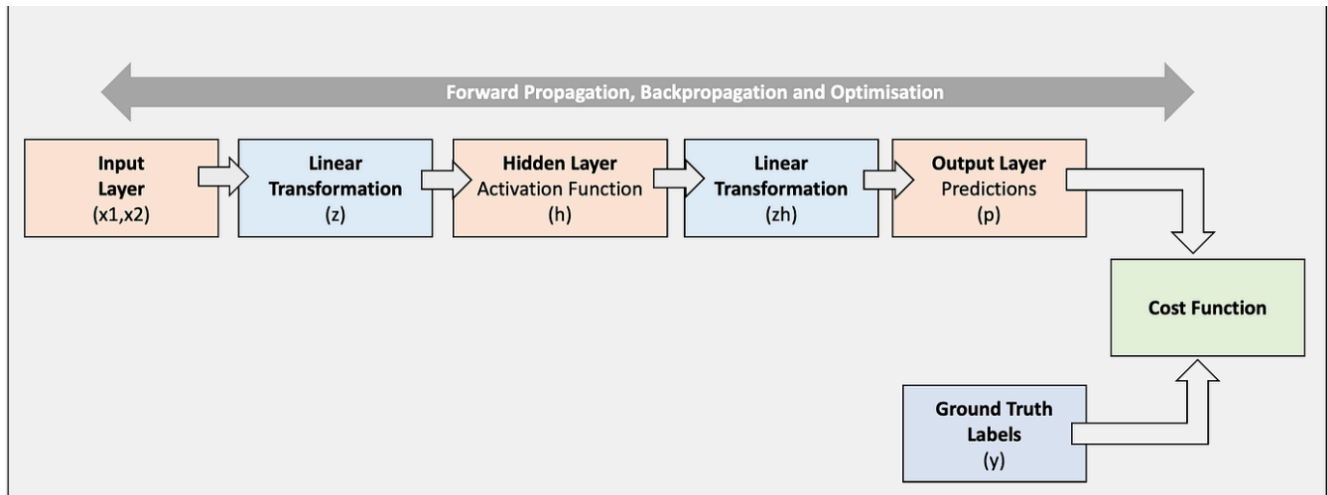
A **Multilayer Perceptron (MLP)** is a feedforward neural network consisting of an input layer, one or more hidden layers, and an output layer. In an MLP with **sigmoid activation functions**, each hidden neuron learns a nonlinear transformation of the input data. Adding more hidden neurons increases the network's **representational power** because each neuron captures different patterns or features from the input. The combined outputs of many neurons allow the network to model highly complex, nonlinear relationships that cannot be represented by a simple linear model. According to the **Universal Approximation Theorem**, an MLP with a sufficient number of hidden neurons can approximate almost any continuous function.

However, increasing the number of hidden neurons **does not always improve performance**. A larger network contains more trainable parameters, making it easier to memorize the training data instead of learning general patterns. This phenomenon is called **overfitting**, where the model achieves very high training accuracy but performs poorly on new, unseen data. Overfitting is especially likely when the training dataset is small or contains noise.

To achieve good **generalization**, the network complexity must be balanced with the amount of available training data. Techniques such as **regularization (L1/L2)**, **dropout**, **early stopping**, and **cross-validation** help prevent overfitting by limiting model complexity and improving performance on unseen data. Increasing hidden neurons also increases computational cost and training time without guaranteeing better accuracy.

Conclusion:

Adding more hidden neurons improves the MLP's ability to learn complex nonlinear functions and increases its representational power. However, beyond an optimal point, additional neurons may cause overfitting, reduce generalization, and increase computational cost. Therefore, selecting an appropriate network size along with proper regularization techniques results in the best balance between **model capacity** and **generalization performance**.



5. Backpropagation, SGD & Curse of Dimensionality

A deep neural network is trained using Stochastic Gradient Descent (SGD) on a very high-dimensional dataset.

Question:

Analyze how the **curse of dimensionality** impacts:

- Gradient updates in backpropagation
- Convergence speed of SGD
- Model generalization

Propose at least two techniques to overcome these issues and justify them analytically.

Answer:

Backpropagation, SGD and the Curse of Dimensionality

The curse of dimensionality refers to the problems that arise when the number of features (dimensions, d) in a dataset becomes very large. In deep neural networks trained using Stochastic Gradient Descent (SGD), high-dimensional data significantly affects gradient computation, convergence, and model performance.

1. Impact on Gradient Updates in Backpropagation

During backpropagation, network parameters are updated as:

where η is the learning rate and ∇L is the gradient of the loss function.

In high-dimensional spaces:

The number of parameters increases rapidly, producing very large gradient vectors.

Some dimensions may contain irrelevant or noisy information, causing unstable gradient directions.

Gradients can become very small (vanishing gradients) or very large (exploding gradients), making learning difficult.

Sparse data distributions lead to inaccurate gradient estimates because samples are far apart in high-dimensional space.

Thus, backpropagation requires more updates to identify the optimal direction.

2. Impact on Convergence Speed of SGD

SGD updates parameters using a small batch of training samples:

In high dimensions:

The optimization landscape becomes more complex with many local minima and saddle points.

Gradient variance increases, causing oscillations.

More parameters require additional iterations and computational cost.

Sparse data reduces the effectiveness of each update.

Hence, SGD converges more slowly and may require adaptive optimizers such as Adam or RMSProp.

3. Impact on Model Generalization

Generalization refers to performance on unseen data.

High-dimensional models often:

Learn noise instead of meaningful patterns.

Have increased risk of overfitting.

Require much larger datasets to represent the feature space adequately.

If the number of features $\diamond$ is much larger than the number of samples $\diamond$, the model complexity increases, leading to poor prediction accuracy on new data.

Techniques to Overcome the Curse of Dimensionality

(a) Dimensionality Reduction

Methods such as Principal Component Analysis (PCA) or autoencoders reduce the number of features:

where $\diamond$ contains important components.

Justification:

Removes redundant features.

Reduces computation.

Produces more stable gradients.

Improves convergence speed.

(b) Regularization

L2 regularization modifies the loss function:

where λ controls model complexity.

Justification:

Prevents overfitting.

Reduces large weights.

Improves generalization.

(c) Feature Selection (Optional)

Selecting only relevant features decreases dimensionality and improves learning efficiency.

Conclusion

The curse of dimensionality negatively affects gradient updates, slows SGD convergence, and reduces model generalization. Techniques such as dimensionality reduction and regularization effectively reduce complexity, stabilize optimization, and improve overall neural network performance.

Curse of Dimensionality

- Theoretically, increasing features should improve performance

